Practica No.4: Validaciones Pydantic

Tecnologías y Aplicaciones en Internet

Docente: Ivan Isay Guerra Lopez

Alumno: Jesus Benjamin Morales Hernandez

S-205

28/02/26

En esta práctica se desarrolló una API utilizando FastAPI junto con Pydantic para implementar validaciones en los datos recibidos por los endpoints.

El objetivo principal fue asegurar que la información enviada al servidor cumpla con reglas específicas antes de ser procesada.

Pydantic permite definir modelos de datos con restricciones, lo que ayuda a prevenir errores, garantizar la consistencia de la información y mantener la integridad de la aplicación.

1. Envío de datos correctos

Cuando se enviaron datos válidos:

El servidor respondió con código 201 (Created).

Se mostró un mensaje confirmando que el usuario fue agregado correctamente.

Esto demuestra que las validaciones permitieron el registro cuando la información cumplía con las reglas establecidas.

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:5000/v1/usuarios/' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:5000/v1/usuarios/
```

Server response

Code	Details
200	Response body


```
{
  "status": 200,
  "total": 3,
  "data": [
    {
      "id": 1,
      "nombre": "benjamin",
      "email": "ben@gmail.com",
      "edad": 25
    },
    {
      "id": 2,
      "nombre": "benjamin",
      "email": "ben@gmail.com",
      "edad": 25
    },
    {
      "id": 3,
      "nombre": "benjamin",
      "email": "ben@gmail.com",
      "edad": 25
    }
  ]
}
```


Download

Response headers

```
content-length: 222
content-type: application/json
date: Sun,01 Mar 2026 00:36:58 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

Media type

application/json

Controls Accept header.

Conclusión

La implementación de validaciones mediante Pydantic en FastAPI permite controlar de manera eficiente los datos que ingresan a la API.

Gracias a estas validaciones, es posible evitar inconsistencias, reducir errores y mejorar la seguridad y confiabilidad del sistema.

Además, se observó la diferencia entre:

Errores de validación automática (422), generados por Pydantic.

Errores de lógica del negocio (400), manejados manualmente en los endpoints.

En conclusión, el uso de Pydantic fortalece el desarrollo de APIs robustas y bien estructuradas.

```
curl -X 'POST' \
  'http://127.0.0.1:5000/v1/usuarios/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '     { "id": '\''asa'\'',
      "nombre": "benjamin",
      "email": "ben@gmail.com",
      "edad": '\''hola'\''
    }'
```

Request URL

```
http://127.0.0.1:5000/v1/usuarios/
```

Server response

Code	Details
422	Error: Unprocessable Entity

Response body

```
{
  "detail": [
    {
      "type": "json_invalid",
      "loc": [
        "body",
        13
      ],
      "msg": "JSON decode error",
      "input": {},
      "ctx": {
        "error": "Expecting value"
      }
    }
  ]
}
```

Download

Response headers

```
content-length: 125
content-type: application/json
date: Sun,01 Mar 2026 00:39:36 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

Media type

application/json

Controls Accept header.

Example Value | Schema